

中山大学

《软件工程》期末大作业

架构设计文档

项目名称：逸仙活动云——中山大学校园活动云平台

小组成员：钟梓俊 邱剑波 张宸睿

指导老师：王可泽

2026 年 7 月 16 日

目录

1	架构目标与原则	1
2	总体架构	1
3	前端架构	1
4	后端架构	2
5	关键设计决策	2
6	安全与质量设计	3
7	部署视图与演化	3
8	质量属性场景	4
9	接口与数据一致性设计	4
10	威胁与边界分析	4

1 架构目标与原则

架构以“可演示、可联调、可部署、可演化”为目标：前端与后端独立构建，职责清晰；鉴权与业务规则在服务端强制执行；外部服务可选并可降级；开发环境零/低配置，部署环境支持容器化。遵循接口优先、依赖反转、最小权限、可观测性和配置外置原则。

2 总体架构



图 1: 系统总体架构——前后端分离，外部服务可选并可降级

前端开发阶段中，Vite 将 /api 请求代理到 127.0.0.1:5000；生产环境建议由反向代理统一承接 TLS、静态资源、API 路由和安全头。后端 Docker Compose 已定义 API、PostgreSQL、Redis、Worker、Beat 以及可选 Copilot 代理服务。

3 前端架构

前端采用分层单页应用：views 承担页面组装和路由入口，components 承担可复用视觉组件，api 统一封装 Axios 请求与数据适配，stores 承担认证等跨页面状态，composables/utils 放置页面逻辑和纯函数。路由守卫基于登录状态和角色元信息进行导航控制；请求拦截器注入 Token，响应拦截器统一处理网络异常和 401/403。页面领域对象维持 Activity，隔离后端实体变更。

4 后端架构

层次	责任	代表实现
应用层	应用工厂、配置加载、扩展初始化、统一错误和请求日志	app/__init__.py、Config、CORS、JWT、SQLAlchemy
接口层	HTTP 路由、参数解析、状态码、身份/角色入口控制	auth、activities/posters、search、calendar、knowledge 等蓝图
服务层	可复用业务规则、内容解析、质量、去重、审计、通知和安全校验	services/*
领域/持久层	实体关系、约束、事务与序列化	User、Poster、Registration、DataSource、AuditLog 等模型
基础设施层	数据库、Redis、Celery、邮件、外部搜索/抓取/AI	extensions、容器和环境变量配置

采用蓝图而不是单一巨型控制器，避免业务逻辑散落在路由中；采用应用工厂支持测试配置与多环境运行；采用服务层隔离质量评分、知识重建和安全 URL 校验等可独立测试的策略。

5 关键设计决策

编号	决策	原因/收益	代价与应对
AD-01	前后端分离，REST/JSON 契约	可独立迭代与部署；Mock 支持前端先行开发。	存在字段命名差异；以契约矩阵、适配层和联调清单控制。
AD-02	JWT + RBAC	无状态认证，明确 viewer/publisher/admin 边界。	API Token 生命周期和撤销需运维；生产环境配置强密钥、HTTPS 和刷新/吊销策略。
AD-03	活动审核状态机 + 审计日志	发布前内容治理，可回溯提交、批准、驳回、合并等操作。	状态增长会复杂；以集中转换规则和状态测试维护。

编号	决策	原因/收益	代价与应对
AD-04	关系型数据库 + 唯一约束	用户、活动、报名、收藏、节点关系事务一致;幂等可由数据库兜底。	迁移管理需完善;采用迁移工具替换生产环境的自动建表。
AD-05	Worker/Beat 异步入口	抓取、AI、定时任务不阻塞 Web 请求。	队列/重试可观测性要求提高;记录任务日志、死信与告警。
AD-06	Docker Compose 部署拓扑	服务依赖可复现,数据库数据卷可持久化。	Compose 不等于高可用;生产可演化到编排平台和托管数据库。

6 安全与质量设计

认证端点使用验证码和限流,密码使用哈希存储;所有受限端点由 JWT 与角色装饰器服务端校验。内容生成时对 HTML 转义;外部抓取校验协议、允许域、回环/内网地址及重定向,降低 XSS/SSRF 风险。搜索日志记录脱敏查询词、耗时、命中数和请求标识;关键审核操作落审计表。API 统一将 HTTP 异常和未处理异常格式化为 JSON,前端以 PageState 和拦截器提供可理解的失败体验。

7 部署视图与演化

开发模式使用 Mock 或本地 Flask + SQLite;部署模式使用 Gunicorn 承载 Flask、PostgreSQL 数据卷、Redis、Celery Worker/Beat。配置通过 .env 注入,仓库保留 .env.example。下一阶段应完成:数据库 schema migration、Nginx/TLS、备份恢复演练、健康探针与指标采集、镜像版本固定、依赖扫描及灰度/回滚流程。上述设计既保留了课程项目可运行的轻量性,也为真实校园使用提供了清晰的架构演化路径。

8 质量属性场景

属性	场景与响应	架构支撑
可修改性	新增活动字段或新搜索筛选时，只修改适配器、DTO、服务/模型和对应页面/测试，不扩散到所有组件。	前后端分层、API 模块、领域名称隔离、服务层。
可用性	Redis、邮件或外部搜索临时不可达时，核心浏览/审核请求不应整体阻塞；返回可理解的降级/错误。	外部能力隔离、超时、统一异常 JSON、Worker 异步入口。
安全性	恶意用户尝试直接调用审核/数据源接口或提交内网抓取 URL。	服务端 RBAC、JWT、限流、URL/重定向安全校验、审计。
可测试性	测试不连接真实数据库、邮件、LLM 或 Redis，却能验证业务分支。	应用工厂、TestConfig、内存 SQLite、依赖配置开关和服务层纯函数。
可部署性	开发者在干净环境以环境样例和 Compose 重建服务。	Dockerfile、Compose、Gunicorn、.env.example、健康检查。

9 接口与数据一致性设计

写操作采用“校验—领域处理—事务提交—返回 DTO”的顺序；活动发布/审核在同一业务边界内更新状态、生成审计记录，并在需要时触发知识/通知服务。对报名、收藏、日历等关系使用数据库唯一约束提供最终幂等保证。前端不得依据按钮隐藏承担授权职责，所有角色边界必须由 API 复核。跨服务异步任务应使用可重试、可观测的消息/队列语义；当异步结果失败时，避免将活动状态提前标为成功。

10 威胁与边界分析

主要信任边界包括浏览器—API、API—数据库/Redis、Worker—外部站点与 API—邮件/AI 服务。针对浏览器输入，采用格式校验、HTML 转义、认证和 CSRF/同源策略配置；针对 Token，采用 HTTPS、短生命周期与安全存储策略；针对抓取，采用协议、域名、内网和重定向校验；针对日志，采用查询词/个人信息脱敏。生产补强项为安全响应头、速率阈值监控、依赖扫描、密钥托管和定期恢复演练。